Real Estate Investment Analyzer

Multi-Agent System for UAE Rental Market

Mohamad — inmind.academy, AI & ML Track, Spring 2026

The Problem

Investors in the UAE rental market need to cross-reference **property listings, market reports, financial calculations, and comparable valuations** before making a decision.

Today that means: searching listing sites, reading PDF reports, doing mortgage math manually, and comparing neighborhoods — **hours of work per property**.

> I built a system where you ask a question like *"Should I invest in Business Bay?"* and get a full analysis in seconds — powered by multiple AI agents working together.

Architecture

5 independent Docker containers. 2 agent frameworks. Real MCP protocol.

System Overview

(Screenshot of architecture diagram recommended here)

5 services, each in its own Docker container:

Container	Framework	Role	Port
Agent System A	LangGraph + FastAPI	Supervisor + 2 specialists	8000
Agent System B	Google ADK + FastAPI	Independent property valuation	8001
MCP Server	fastmcp (SSE)	Mortgage + tax calculators	8002
Qdrant	Vector DB	PDF semantic search	6333
Frontend	React + Vite + Nginx	Chat UI with voice + streaming	5173

Agent A calls Agent B via **HTTP POST** (not a Python import). Agent A connects to MCP via **real MCP protocol** (SSE transport with capability negotiation).

Agent System A — How Routing Works

The supervisor uses **structured output** (`with_structured_output`) to decide who handles each query:

```
class RouteDecision(BaseModel):
    route: Literal["property_analyst", "market_researcher", "both", "direct"]
    summary: str  # "Searching Sharjah apartments ... " — zero extra LLM call
```

Route	When	Tools Available
`property_analyst`	Property search, prices, mortgage	RAG + web + MCP (mortgage, tax)
`market_researcher`	Trends, comparisons, investment	RAG + web + Agent B valuation
`both`	Complex queries needing both	Parallel via ThreadPoolExecutor
`direct`	Greetings	No tools

Why structured output? OpenRouter/Claude supports `method="function_calling"` natively — zero parsing errors. The `summary` field gives the frontend a dynamic thinking status for free.

Agent System B — Pipeline State Pattern

The problem: LLMs mangle complex JSON when passing data between tool calls.

My solution: Each tool stores its result in a shared `_pipeline_state` dict. Steps 3 and 4 take **zero arguments** — they read from previous steps directly.

Step	Tool	Args	What It Does
1	`tool_parse_request`	location, type, beds	Parse + validate criteria
2	`tool_find_comparables`	*(none — reads step 1)*	CSV filter + Qdrant search
3	`tool_evaluate_comparables`	*(none — reads step 2)*	LLM scores comps, estimates value
4	`tool_synthesize_response`	*(none — reads step 3)*	Build final response

Result: Eliminated ~20% of Agent B failures caused by serialization errors.

Agent B logs each step: `Step 2: Found 8 comparables | market_context_chunks=3`

RAG Pipeline — Design Decisions

Two data sources, handled differently — on purpose:

Data	Method	Why
CSV listings	**Pandas filters**	Structured data needs exact filtering, not semantic similarity
PDF market reports	**Qdrant vector search**	Unstructured text needs semantic matching

Chunking

- **2000 characters (~512 tokens)** with 200-char overlap
- `RecursiveCharacterTextSplitter` uses characters, not tokens — I initially had 512 chars (~128 tokens) which was too small and split paragraphs. Fixed to 2000 chars after testing.

Embedding

- **text-embedding-3-small** (1536 dims) — good quality at $0.02/1M tokens
- `text-embedding-3-large` gave <2% MRR improvement at 6x cost — not worth it

MCP Server — Implementation

Built with `fastmcp` using SSE transport. Agent A connects via the `mcp` SDK — not REST.

Tool	What It Computes	UAE-Specific Logic
`calculate_mortgage`	Monthly payment, total cost, DTI	20% down, 4.5% rate, 25yr defaults
`estimate_property_tax`	Housing fee, municipality fee, registration	Dubai 5%, Abu Dhabi 3%, Sharjah 2%

Why MCP instead of regular Python functions?

- Real protocol with capability negotiation and tool discovery
- If I add a new tool to the MCP server, Agent A discovers it automatically at runtime
- Demonstrates understanding of the standard (not just a REST wrapper)

Guardrails — 2-Layer Defense

Input Guardrails

- **Layer 1 (Regex):** 10+ injection patterns — "ignore your instructions", "DAN mode", ChatML injection, system prompt extraction. Runs in <1ms.
- **Layer 2 (ML):** DeBERTa-based prompt injection scanner via LLM Guard. Catches subtle attacks regex misses.
- **Graceful degradation:** If LLM Guard not installed, falls back to regex-only.

Output Guardrails

- **Price hallucination check:** Flags rent values outside realistic AED ranges
- **Empty/short response detection:** Catches agent failures before reaching the user
- **PII scanning:** Regex patterns for sensitive data leakage

Both layers run on every request in the FastAPI endpoint — **before** the supervisor (input) and **after** the response is generated (output).

Voice Interface

Speech-to-Text

- **Groq Whisper** `large-v3-turbo` API
- **Multilingual auto-detection** — works with English, Arabic, French (no hardcoded language)
- Frontend records via MediaRecorder, sends blob to `/transcribe`

Text-to-Speech

- **Browser Web Speech API** — instant playback, zero cost, no external API needed
- Markdown stripped before reading (removes code blocks, headers, links)

End-to-End Flow

```
Microphone → MediaRecorder → /transcribe (Groq Whisper) → text
   → Agent pipeline → response text → speechSynthesis → Speaker
```

Docker Deployment

```
docker compose up --build -d   # One command — starts all 5 containers
```

Startup order: Qdrant (+ health check) -> MCP Server + Agent B -> Agent A -> Frontend

Auto-ingestion: On startup, Agent A checks if Qdrant has data. If empty, runs the full PDF -> chunk -> embed -> store pipeline automatically. **Zero manual steps.**

(Screenshot of `docker ps` showing 5 containers recommended here)

Feature	Implementation
Health checks	Qdrant TCP probe, dependency ordering
Session management	In-memory dict with 1-hour TTL + periodic cleanup
Error handling	Try/catch on all external calls, graceful fallback messages
Cost tracking	Per-model token counting via `/costs` endpoint

Live Demo

5 queries across different routes + 1 failure case.

Demo Queries

#	Query	Route	What to Watch
1	"Find 2-bed apartments in Dubai Marina under 80k"	`property_analyst`	CSV search, area stats
2	"What are the rental market trends in Sharjah?"	`market_researcher`	PDF RAG + web search
3	"Compare JBR vs Marina for yield + calculate mortgage"	`both`	Parallel routing + MCP
4	"Should I invest in Business Bay?"	`market_researcher`	Agent B valuation
5	"Hello!"	`direct`	No tools — greeting

Failure case: "Find cheapest apartment in Al Ain" — sparse data, Agent B timeout

(Live demo here — show Docker containers, then run queries in the UI)

Evaluation

20 test cases. Retrieval + generation metrics. 2 config comparisons. 3 failure cases.

Evaluation Results

Retrieval (10 PDF-sourced questions)

Metric	Score	Rating
Precision@5	0.640	Good — 64% of chunks relevant
Recall@5	0.527	Acceptable — room for improvement
MRR	0.833	Good — first relevant chunk usually rank 1

Generation (LLM-as-Judge, 20 questions, 1-5 scale)

Metric	Score
Correctness	2.20 / 5
Faithfulness	3.10 / 5
Relevance	2.85 / 5
Completeness	2.20 / 5
Overall	**2.59 / 5**
Avg Latency	7.06s

Configuration Comparisons

Top-K Retrieval: K=3 vs K=5

Metric	K=3	K=5	Winner
Precision	0.667	0.640	K=3
Recall	0.342	**0.527**	**K=5**
MRR	0.833	0.833	Tie

Conclusion: K=5 wins — higher recall means less missing context, which reduces hallucination.

Chunk Size: 256 vs 512 tokens (simulated)

Metric	256 tokens	512 tokens	Winner
Precision	0.580	**0.640**	**512**
Recall	0.342	**1.000**	**512**
MRR	0.803	**0.833**	**512**

Conclusion: 512 dominates. 256 splits paragraphs at sentence boundaries —

Failure Analysis

Failure 1: "Find 2-bed apartments under 80k in Dubai Marina"

- **Score:** 1.0/5 | **Component:** Generation
- **Root cause:** Agent returned a greeting instead of property results — routing or session issue
- **Fix:** Add fallback detection — if response doesn't contain expected data format, retry

Failure 2: "Calculate mortgage for 1,500,000 AED property"

- **Score:** 1.0/5 | **Component:** MCP connection
- **Root cause:** MCP server call failed silently — agent responded without mortgage data
- **Fix:** Make MCP tool failure explicit in the response ("mortgage calculation unavailable")

Failure 3: "Find cheapest apartment in Al Ain"

Key Design Decisions

Decision	Why	Alternative I Rejected
CSV via pandas, not Qdrant	Exact filters on structured data	Embedding rows — semantic search too imprecise
Pipeline state dict (Agent B)	LLMs can't reliably pass JSON between tools	Letting LLM serialize — 20% failure rate
`openrouter/auto`	Cost-efficient, auto-picks best model	Fixed Claude Sonnet — 3x cost
2-layer guardrails	Fast regex + thorough ML, graceful fallback	ML-only — too slow; regex-only — too shallow
MCP over SSE	Real protocol with tool discovery	REST wrappers — no standard
Groq Whisper API	Free tier, fast, multilingual	Local Whisper — slow on CPU
2000-char chunks (~512 tokens)	Captures full paragraphs	512 chars (~128 tokens) — split mid-sentence

What I Would Do Differently

- **Hybrid search** (BM25 + vector) — would catch exact names like "Dubai Marina" that vector search misses. This is why Q2 failed.
- **Redis for sessions** — current in-memory dict doesn't survive restarts
- **Cross-encoder re-ranking** — would improve precision by 10-15%
- **Streaming from Agent B** — currently blocks until full pipeline completes
- **Table-aware PDF parsing** — would fix failures on questions about specific numbers in report tables
- **Fine-tuned BERT** for intent classification — faster and cheaper than LLM routing

Thank You

Questions?

Mohamad — inmind.academy, AI & ML Track, Spring 2026

Real Estate Investment Analyzer — UAE Rental Market